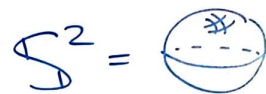
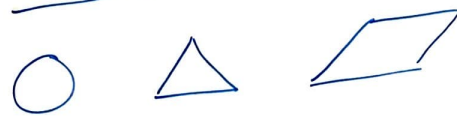


Q1: What is Shape?

- spaciality
- the way an object occupies space
- formulas for area, perimeter, ...
- State of something: in shape?
what shape is the thing?
- enclosed boundary
- invariant under scaling
- has points
- has properties: path-connected, χ s,
distances, projections
 ↳ geometric (see above)
 ↳ topological (eq. path-connected)

Some Shapes:



G. Carlsson

"Data has shape, and that shape matters"

Q2: What is the shape of data?

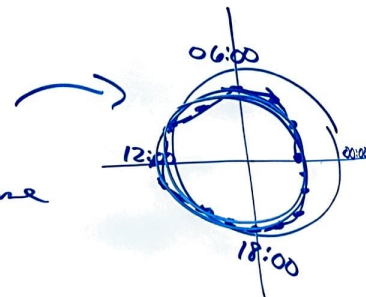
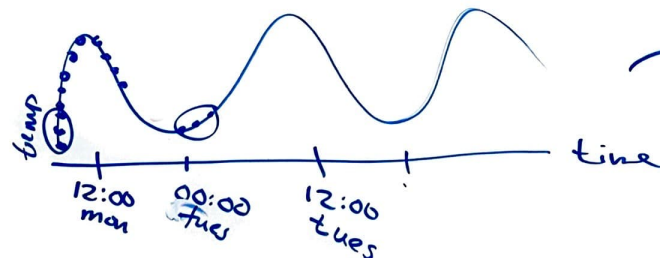
- Clustering $\leftarrow$ 0-dimensional persistence, dendrograms
- need distances defined
- interesting shapes need data that varies
- PCA: Reduces dimensionality, preserving shape
- can describe w/ statistics: mean, variance, etc.

+ descriptors: persistence

• "underlying shape", sampling from it
→ determine what the parameters were

- distributions, often parameterized

- example: "weather is cyclic" "circle"



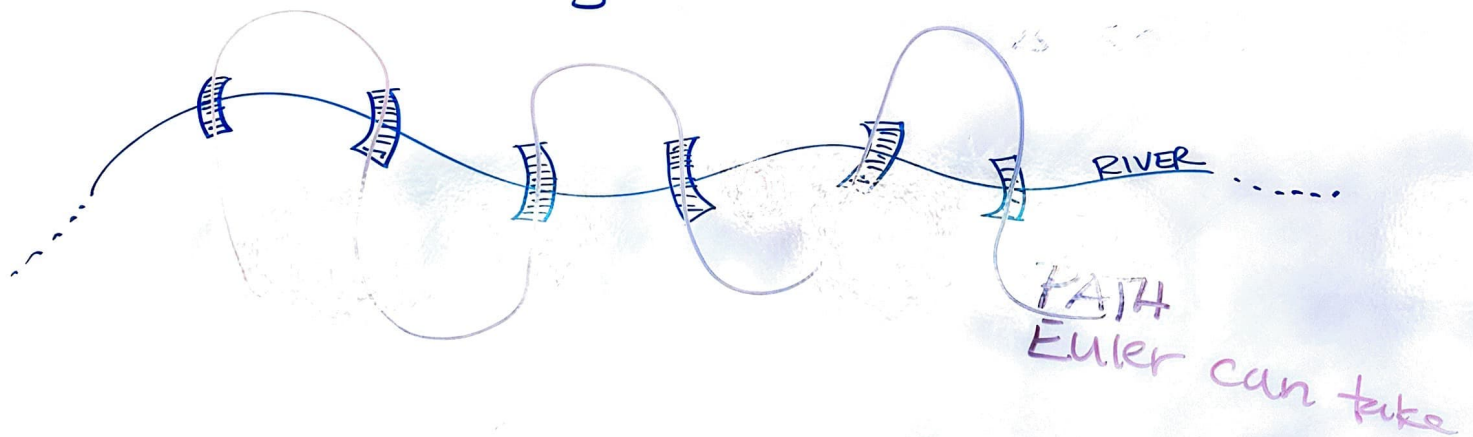
other opt: Sliding Window Embedding (SWE)

- Point Cloud Data: connect the dots!
U-Rips cplx, Čech cplx "neighborhood complexes"

Shape (BTF)

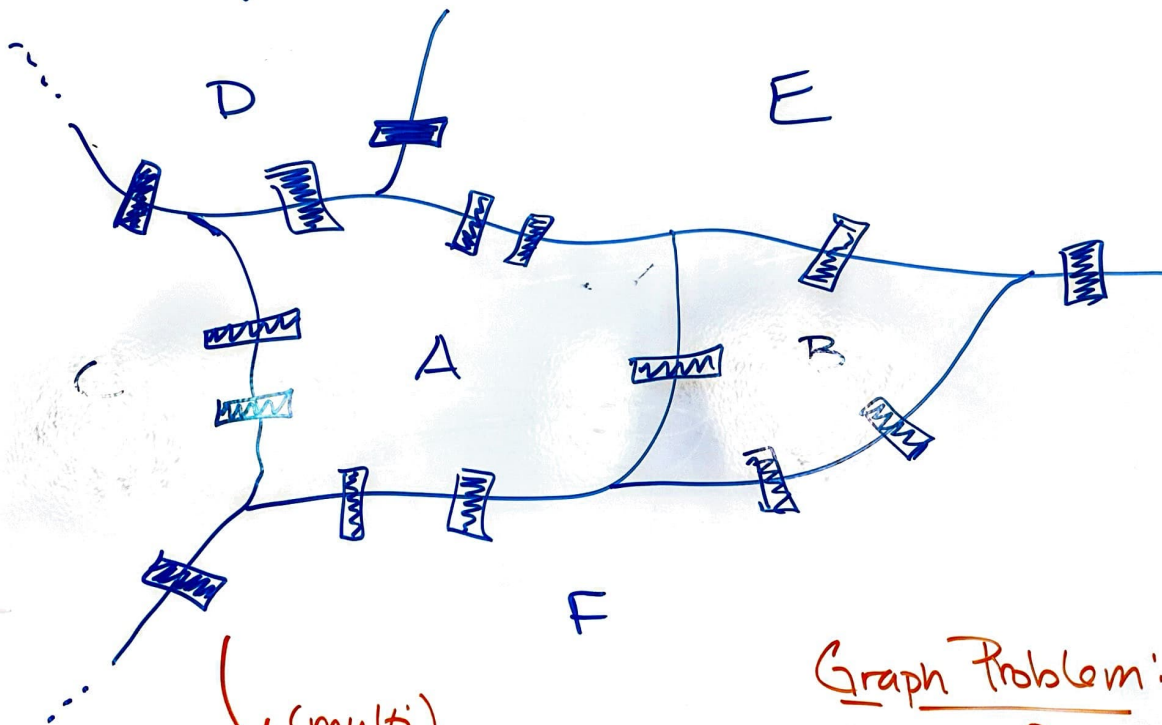
The meaning that we (as people) attribute to a set (or data) that helps to explain connections between elements of that set.

Question: Can I go on a walk
and cross each bridge
exactly once?

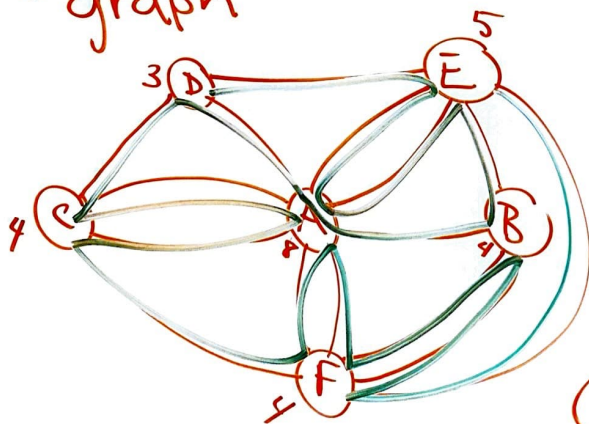


What about the cities in
fig 1 + fig 3 ?
Königsberg

Fig 3.



(multi)
graph



Graph Problem:

Can I find a walk
through the graph that
covers each edge exactly
once?

↳ Eulerian tour

(Hamiltonian: each vertex
exactly once)

Theorem:

An Eulerian tour exists
iff there are exactly
{either 2 or 0} vertices of
odd degree, and the
graph is connected.

(Annotations: An arrow from '2' points to 'tells us where to start/stop'. An arrow from '0' points to 'start/stop anywhere'.)

[Q] Now I know one exists.
How do I find it?

Algorithm (Euler Tour)

Input: connected graph $G=(V,E)$

Output: a path p that is an E.T.

1: If $\# \text{ odd deg} \neq 0 \text{ or } 2$

2: fail!

3: $a, b \leftarrow$ ^{valid} start/end vertices

4: $p = (p_1, p_2, \dots, p_n)$ _{$\begin{smallmatrix} a \\ b \end{smallmatrix}$} via WFS, marking edges

5: while $\exists$ unmarked edges

6: | $e = (c, c')$, where $c \in p$

7: | $q = (q_1, q_2, \dots, q_k)$ _{$\begin{smallmatrix} c \\ c' \end{smallmatrix}$} via WFS through unmarked edges, marking as we go.

8: | stitch q into p

9: endwhile

10: return p

